



Linear Regression & Gradient Descent

Brief Overview

This note covering linear regression and gradient descent was created from the [Stanford CS229: Machine Learning - Linear Regression and Gradient Descent | Lecture 2 \(Autumn 2018\)](#) YouTube video. The 78-minute lecture provides a thorough walkthrough of the theory and practical implementation of linear models and their optimization.

It covers **hypothesis representation**, **cost function**, gradient descent, normal equations, and practical implementation tips.

Key Points

- Linear hypothesis as a weighted sum of features.
 - Squared-error cost function and its properties.
 - Gradient descent update rule and learning-rate considerations.
 - Closed-form solution via normal equations and when to use it.
-



Linear Regression Overview

- **Supervised learning** regression problem: map input features X to a continuous output Y (e.g., house price).
- Example: predict house prices from size (sq ft) and number of bedrooms.



Key Notation

Symbol	Meaning
m	Number of training examples
n	Number of <i>features</i> (excluding the bias term)
$x^{(i)}$	Feature vector for the i -th training example
$y^{(i)}$	Target (price) for the i -th training example
$x_j^{(i)}$	Value of feature j in example i

$x_0^{(i)}$	Dummy feature, always 1 (bias)
θ_j	Parameter (weight) for feature j
θ	Parameter vector $(\theta_0, \theta_1, \dots, \theta_n)^T$
$h_\theta(x)$	Hypothesis (prediction) given parameters θ and input x
$J(\theta)$	Cost function (measure of prediction error)
α	Learning rate for gradient descent



Hypothesis Representation

Definition: The hypothesis is a linear (affine) function of the features.

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

Using the dummy feature $x_0 = 1$, this compactly becomes

$$h_\theta(x) = \sum_{j=0}^n \theta_j x_j$$



Cost Function (Ordinary Least Squares)

Definition: The cost function quantifies the squared prediction error over the training set.

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

- The factor $\frac{1}{2}$ simplifies derivative calculations.
- Squared error is chosen because it leads to a convex objective and corresponds to a Gaussian noise model.



Gradient Descent

Intuition

- Imagine standing on the surface defined by $J(\theta)$.

- At each step we look in the direction of steepest descent (the **gradient**) and take a small step to lower the cost.

Update Rule

For each parameter θ_j (including $j = 0$):

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Deriving the Partial Derivative

$$\frac{\partial}{\partial \theta_j} J(\theta) = \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Thus a single gradient-descent step becomes

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- **Learning rate** α controls step size (common default 0.01; adjust after feature scaling).
- **Initialization** can be all zeros or random values; for linear regression the algorithm converges to the global minimum regardless of start point because $J(\theta)$ is convex.

Batch vs. Stochastic Gradient Descent

- **Batch GD** uses the sum over *all* m examples each iteration (as above).
- **Stochastic GD** updates θ after each individual example, providing faster but noisier convergence.



Normal Equations (Closed-Form Solution)

- An alternative to iterative GD: solve θ analytically by setting the gradient to zero.
- Gives the exact minimizer in one step, efficient when n is small and the data matrix is invertible.



Additional Concepts

- The **learning algorithm** takes the fixed training set and outputs a **hypothesis** h_{θ} that can predict prices for unseen houses.
- Choosing **parameters** θ that minimize $J(\theta)$ yields the best fit under the squared-error criterion.



Convexity & Contour Visualization

Quadratic Cost Landscape

Definition: $J(\theta)$ is a quadratic (convex) function, producing a bowl-shaped surface.

- The bowl has **no local optima** other than the global optimum.
- Contour lines of constant cost are **ellipses** (circles if features are equally scaled).

Gradient Direction

Fact: The steepest-descent direction is orthogonal to contour lines.

- Each gradient-descent step moves perpendicular to the current ellipse, descending toward the minimum.



Learning Rate α

Effects of α

- **Too large** $\rightarrow$ steps overshoot, $J(\theta)$ may increase.
- **Too small** $\rightarrow$ convergence is very slow, requiring many iterations.

Practical Tuning

- Test values on an exponential (doubling) scale, e.g., 10^{-1} , 10^{-2} , 10^{-4} , 10^{-8} , etc.
- Choose the α that drives $J(\theta)$ down fastest **without** causing the cost to rise.



Visualizing Gradient Descent on Data

- Dataset contains $m = 49$ training examples.
- Initial parameters $\theta_0 = 0$, $\theta_1 = 0$ give a horizontal hypothesis line ($\hat{y} = 0$).
- Each iteration updates θ , rotating and shifting the line to better fit the points.
- After many iterations the line converges to the optimal straight-line fit.



Batch vs. Stochastic Gradient Descent

Batch Gradient Descent

Definition: Uses the sum over **all** m examples to compute each parameter update.

- One update = one full pass through the dataset.
- Becomes expensive when m is very large (e.g., millions of examples).

Stochastic Gradient Descent (SGD)

Definition: Updates parameters after processing a **single** training example.

- Inner loop: for $i = 1$ to m , update each θ_j using the gradient from example i .
- The trajectory is noisy but, on average, moves toward the global minimum.
- Parameters keep oscillating and never settle exactly, yet converge sufficiently for prediction.

Mini-Batch Gradient Descent

- Processes a small batch (e.g., 100 examples) per update, blending the stability of batch GD with the speed of SGD.

Choosing an Approach

Dataset size	Recommended method	Reason
Small (hundreds-thousands)	Batch GD	Simpler; no oscillation
Large (millions-plus)	SGD or Mini-batch	Avoids full-dataset scan each step
Extremely large (terabytes)	SGD with decreasing α	Scalable; still yields good predictions

Controlling SGD Convergence

- **Learning-rate schedule:** Gradually reduce α ; smaller steps diminish oscillations.
- **Stopping criterion:** Monitor $J(\theta)$ over iterations and stop when it no longer decreases.

Matrix Calculus Essentials

Vector- and Matrix-Derivatives

- For a scalar function $J(\theta)$ with $\theta \in \mathbb{R}^{n+1}$,

$$\frac{\partial J}{\partial \theta}$$

is an $(n + 1) \times 1$ column vector of partial derivatives.

- For a matrix $A \in \mathbb{R}^{p \times q}$ and scalar function $f(A)$,

$$\frac{\partial f}{\partial A}$$

is a $p \times q$ matrix where each entry is the derivative of f with respect to the corresponding element of A .

Trace Operator

Definition: $\text{tr}(A) = \sum_i A_{ii}$ (sum of diagonal entries).

Properties:

- $\text{tr}(A) = \text{tr}(A^T)$.
- **Cyclic property:** $\text{tr}(AB) = \text{tr}(BA)$; more generally $\text{tr}(ABC) = \text{tr}(BCA) = \text{tr}(CAB)$.
- **Derivative rule:** If $F(A) = \text{tr}(AB)$ with constant B , then

$$\frac{\partial F}{\partial A} = B^T.$$

These tools are used to derive the **normal equation**, giving a closed-form solution for θ in linear regression without iterative optimization.

Matrix Form of Linear Regression

Definition: The *design matrix* $X \in \mathbb{R}^{m \times (n+1)}$ stacks the m training examples as rows, each row being the transposed feature vector $x^{(i)T}$ (including the bias feature $x_0^{(i)} = 1$).

Symbol	Meaning
--------	---------

X	Design matrix, rows = training examples, columns = features (including bias)
y	Column vector $\in \mathbb{R}^m$ of target values $y^{(i)}$
θ	Parameter column vector $\in \mathbb{R}^{n+1}$
$h_{\theta}(x^{(i)})$	Prediction = $x^{(i)T}\theta$

The hypothesis for **all** examples can be written compactly as

$$X\theta; =; \begin{bmatrix} x^{(1)T}\theta \\ x^{(2)T}\theta \\ \vdots \\ x^{(m)T}\theta \end{bmatrix}$$

The vector of **prediction errors** is

$$e; =; X\theta - y; =; \begin{bmatrix} h_{\theta}(x^{(1)}) - y^{(1)} \\ \vdots \\ h_{\theta}(x^{(m)}) - y^{(m)} \end{bmatrix}$$

Using the identity $z^T z = \sum_{i=1}^m z_i^2$, the **cost function** becomes

$$J(\theta); =; \frac{1}{2}, (X\theta - y)^T (X\theta - y)$$

Thus $J(\theta)$ is exactly one-half the sum of squared errors, expressed without an explicit summation.



Deriving the Normal Equations



Analogy: The scalar derivative $\frac{d}{dx}(x^2c) = 2xc$ mirrors the matrix rule $\frac{\partial}{\partial A}, \text{tr}(A^2C) = 2AC$.

Step-by-step outline

1. Write $J(\theta)$ in trace form:

$$J(\theta) = \frac{1}{2} \text{tr}[(X\theta - y)^T (X\theta - y)]$$

2. Expand the quadratic expression (similar to $(a - b)^2 = a^2 - 2ab + b^2$):

$$(X\theta - y)^T (X\theta - y) = \theta^T X^T X\theta; -; \theta^T X^T y; -; y^T X\theta; +; y^T y$$

3. Differentiate each term w.r.t. θ using matrix-derivative identities (e.g.,

$$\frac{\partial}{\partial \theta}, \theta^T A\theta = (A + A^T)\theta, \text{ and } A \text{ is symmetric here):}$$

$$\frac{\partial J}{\partial \theta} = X^T X \theta; -; X^T y$$

4. Set the gradient to zero for optimality:

$$X^T X \theta; =; X^T y$$

5. Solve for θ (provided $X^T X$ is invertible):

$$; \theta; =; (X^T X)^{-1} X^T y;$$

These equations are known as the **normal equations** and yield the global minimum of $J(\theta)$ in a single linear-algebra step.

Practical Considerations

Issue	Typical cause	Remedy
$X^T X$ non-invertible	Redundant / linearly dependent features (e.g., duplicated columns)	- Remove or combine dependent features - Use the pseudo-inverse: $\theta = (X^T X)^\dagger X^T y$ (or $\theta = X^\dagger y$)
Very large n (many features)	Computational cost of matrix inversion	- Apply regularization (ridge regression) - Use iterative methods (gradient descent) instead of normal equations

The pseudo-inverse $\dagger$ gives the least-squares solution even when $X^T X$ is singular, but the most reliable fix is to ensure the feature set is free of exact duplicates.

Choosing a Learning Rate α (for GD alternatives)

Empirical guideline: Test values on an exponential scale (e.g., 10^{-1} , 10^{-2} , 10^{-4} , 10^{-8}) and pick the largest α that consistently decreases $J(\theta)$ without causing divergence.

- Too large $\rightarrow$ cost may increase or oscillate.
- Too small $\rightarrow$ convergence becomes excessively slow.

This trial-and-error approach is the standard practical method for setting α when using gradient-descent variants instead of the normal equations.